ACADEMIC PROJECT GUIDE & VIVA HANDBOOK

StayEase — Smart Accommodation Discovery & Booking Platform

Complete end-to-end breakdown of what we built, which technologies were used for what, and how to confidently explain the project to professors and interviewers.

PROJECT TYPE
Full-Stack Enterprise Web App

PRIMARY STACK
Java 21, Spring Boot 3.3.4, PostgreSQL

FRONTEND & MAPS
Thymeleaf, Bootstrap 5, Leaflet.js

GEOSPATIAL APIS
OpenStreetMap, Overpass QL, Photon

DEPLOYMENT
Docker Container on Railway Cloud

TEST VERIFICATION
329 / 329 Automated Tests Passed

1. What is StayEase in Simple Words? (Project Elevator Pitch)

If your professor asks: *"In two minutes, explain what StayEase is and what problem it solves?"*, here is the clear, simple explanation:

The 2-Minute Answer:
"StayEase is a smart web platform built primarily for college students, interns, and young working professionals who move to new cities and struggle to find reliable hostels, PGs, and budget hotels. Most current solutions either charge heavy broker commissions or only show properties in a single hardcoded locality. StayEase solves this in two powerful ways:

1. **Direct Zero-Brokerage Booking:** Students can browse verified StayEase properties, check real-time room availability, and book rooms instantly with automated receipt generation and no brokers involved.
2. **Global Location Intelligence (Worldwide Map):** Wherever the user travels in the world (e.g. Jaipur, Kukas, Delhi, Bengaluru, or abroad), the app detects their device GPS coordinates or lets them type any address with Uber-style autocomplete. It then dynamically scans the physical world up to 100 km using free OpenStreetMap geospatial intelligence to display all nearby Hostels, Hotels, and PGs with exact distances and directions."

2. Before vs. After: What Did We Start With & What Did We Build These Days?

Here is the exact progression of what the project originally was and the major engineering upgrades we implemented step-by-step:

Component	Where We Started (Initial State)	What We Built in These Days (Upgraded State)
Geographic Scope	Locked strictly to sample properties in Bengaluru. Searching other cities returned nothing.	**Worldwide (Anywhere on Earth)** — Searches any city, village, landmark, or GPS coordinates globally.
GPS & Geolocation	Browser GPS was blocked because the server security policy set `Permissions-Policy: geolocation=()`.	Enabled `geolocation=(self)` in Spring Security + added dual-mode fallback (hardware GPS with silent IP network backup).
External POI Discovery	No real-world map data. Only fixed database records.	Integrated 100% free OpenStreetMap Overpass & Nominatim engines to detect real registered hostels, hotels, and PGs live.
Search Experience	User had to type exact coordinates or exact spelling without suggestions.	**Cab / Google Maps Style Autocomplete** — Fast typeahead dropdown with instant address & place-type suggestions.
Search Radius	High ranges (50 km - 100 km) timed out with 504 errors on public geospatial mirrors.	**High-Performance Bounding Box Query** — Returns up to 40 verified stays across 100 km in ~2 seconds.
UI, Stacking & Contrast	Navigation bar was cluttered, dropdowns were cut off by parent card overflow, and banner headers had dark text on dark blue.	Streamlined navbar, fixed `z-index` & overflow stacking contexts, and ensured high-contrast bright white typography.
Cloud Deployment	Only ran on localhost.	Containerized with Docker and deployed live on Railway cloud with automated GitHub CI/CD synchronization.

3. Tech Stack Breakdown: What Technology Was Used for What & Why?

Your professor will definitely ask why you chose each tool. Here is the clear breakdown in simple, understandable terms:

BACKEND **Java 21 LTS + Spring Boot 3.3.4**

What it does: Forms the core engine of our application. Handles all web requests (URLs), business logic, database queries, and security.
Why we used it: Java 21 is a Long-Term Support (LTS) release known for rock-solid stability and speed. Spring Boot provides an enterprise-grade ecosystem where configuration is fast, dependency injection keeps code clean, and built-in Tomcat server allows effortless running.

SECURITY **Spring Security 6 + BCrypt Password Encoder**

What it does: Manages authentication (login/logout), role-based authorization (Customer vs. Admin permissions), session cookies, and CSRF protection.
Why we used it: Passwords are never stored in plain text. Spring Security hashes them using BCrypt with cryptographic salt. It also protects against common attacks like Cross-Site Request Forgery (CSRF) and Cross-Site Scripting (XSS).

DATABASE **PostgreSQL (Production) + H2 (Dev/Testing) + Spring Data JPA (Hibernate)**

What it does: Stores users, rooms, bookings, favorites, maintenance logs, and audit logs. Spring Data JPA acts as the Object-Relational Mapping (ORM) bridge.
Why we used it: Instead of writing error-prone SQL strings in Java, we interact directly with Java Entity objects (`Room` , `Booking` , `User`). PostgreSQL on Railway provides ACID-compliant production reliability, while in-memory H2 allows running all 329 tests in seconds.

FRONTEND **Thymeleaf + Bootstrap 5.3 + Vanilla CSS + ES6 JavaScript**

What it does: Thymeleaf is the server-side template engine that populates HTML files with Java data before sending them to the browser. Bootstrap 5.3 provides a responsive grid layout. Custom Vanilla CSS (`app.css`) provides the luxury modern theme. JavaScript handles client-side events like typeahead and GPS.
Why we used it: Avoids the complexity of managing a separate frontend Node/React server while ensuring fast initial page loads, excellent SEO, and clean HTML rendering.

MAPS & GEO **OpenStreetMap + Leaflet.js + Komoot Photon API (100% Free & Open-Source)**

What it does:

- **Leaflet.js:** Lightweight JavaScript library that renders the interactive map tiles and colored pins on the web page.
- **Komoot Photon API:** Provides the fast Uber-style typeahead autocomplete as the user types a city or locality.
- **OpenStreetMap Overpass & Nominatim:** Scans real geographic areas for hostels, hotels, PGs, and guest houses.

Why we used it: Google Maps Platform charges heavy credit card fees per 1,000 requests. OpenStreetMap and Photon provide worldwide coverage **completely free of cost** with zero API key or billing risk.

DEVOPS **Docker + Apache Maven + Railway Cloud + GitHub**

What it does: Maven manages dependencies and compiles code. Docker packages the Java application, dependencies, and runtime into an isolated container image. Railway automatically builds and deploys this Docker container from GitHub on every git push.
Why we used it: Guarantees that "if it works on my laptop, it works in the cloud" with zero environment discrepancies.

4. System Architecture: How Data Flows in StayEase

When a user opens the browser and searches for accommodations, here is the exact journey of that request:

Step-by-Step Request Flow:

1. **User Interaction:** The user types *"Kukas, Jaipur"* in the search box. The JavaScript debounces the keystrokes (waits 220ms) and calls `/api/discovery/suggest?query=kukas`.
2. **Suggest API:** `DiscoveryApiController` calls `LocationDiscoveryService`, which queries the Photon geocoding API and returns a JSON list of matching locations with latitudes and longitudes.
3. **Selection & Submit:** The user selects a suggestion. The hidden form fields for `lat` and `lng` are filled, and the form submits to `/discovery?lat=27.04&lng=75.89&radius=15`.
4. **Backend Intelligence:**
   - `RoomRepository` queries local PostgreSQL database rooms within the chosen radius using Haversine distance calculations.
   - `OpenStreetMapPlacesProvider` concurrently queries the OpenStreetMap Overpass/Nominatim engine for registered external hotels, hostels, and PGs in that exact coordinate bounding box.
5. **Response Synthesis:** The service merges verified StayEase rooms and external POIs, sorts them by distance from the user, and bundles them into a unified `DiscoveryResponse` DTO.
6. **Render:** Thymeleaf populates the HTML cards and Leaflet.js pins on the server, sending a complete, interactive web page to the browser.

5. Top 12 Viva & Professor Questions (With Winning Academic Answers)

Here are the top questions external examiners and professors ask for this project, along with the exact responses to give:

Q1 Why did you use OpenStreetMap instead of Google Maps API?

"Google Maps requires mandatory credit card billing and charges significant fees per API request, which is unsuitable for an academic project and creates financial risk. We chose **OpenStreetMap (Overpass & Nominatim)** combined with **Photon** because they are open-source, community-driven, offer worldwide coverage, and provide complete programmatic access to raw geographical tags (like `tourism=hostel` or `amenity=student_accommodation`) completely free of cost."

Q2 How does StayEase prevent two users from double-booking the same room?

"We handle this through database-level transactional integrity and concurrency control in Spring Data JPA. When booking a room, the booking service method is marked with `@Transactional(isolation = Isolation.READ_COMMITTED)`. Before confirming, the service verifies that no overlapping active booking exists for that specific room ID in the requested date range. If an overlapping booking is detected, the transaction rolls back with an idempotency check, preventing race conditions."

Q3 What is the difference between Spring MVC, Spring Boot, and Spring Data JPA?

"**Spring MVC** is the web framework that handles HTTP routing, controllers, and model-view dispatching. **Spring Boot** is the overarching framework that eliminates boilerplate XML configuration, auto-configures dependencies, and bundles an embedded Tomcat server. **Spring Data JPA** is the data access layer built on top of Hibernate that lets us perform database operations using Java interfaces (like `JpaRepository`) without writing raw SQL queries."

Q4 How does the typeahead autocomplete work without sending too many requests?

"We implemented **client-side debouncing** with a 220-millisecond delay. When a user types, a JavaScript timer resets on every keystroke and only fires the HTTP fetch request after the user pauses typing for 220ms. In addition, we maintain a client-side JavaScript memory cache and a backend server-side cache (`ConcurrentHashMap`) so duplicate queries (like typing and deleting 'jaipur') return instantly from memory without hitting external APIs."

Q5 How do you store and secure user passwords?

"Passwords are never stored in plaintext. We use Spring Security's **BCryptPasswordEncoder**, which uses an adaptive one-way hashing function with a cryptographic work factor (strength = 10) and an automated random salt for every user. Even if two users have the identical password, their stored hashes in the database are completely distinct, protecting against rainbow table attacks."

Q6 How do you calculate distances between the user's location and accommodations?

"We implemented the **Haversine Formula** in Java (`calculateDistanceKm(lat1, lon1, lat2, lon2)`). Because the Earth is a sphere, simple Euclidean distance (Pythagorean theorem) is inaccurate for coordinates. The Haversine formula calculates the great-circle distance between two points on a sphere of radius 6,371 km using spherical trigonometry."

Q7 **How did you solve the 100 km radius search timeout problem?**

"Initially, querying public Overpass instances with multiple union statements across a 100 km radius circle ($31,415 ext{ km}^2$) exceeded execution memory thresholds and timed out after 6 seconds. We re-engineered the search provider to adopt a **dual-tier architecture**: for radii ≤ 35 km, it runs an optimized single-regex Overpass query; for large radii (50 to 100 km), it switches to a bounded viewbox query via Nominatim. This reduced response times from 18+ seconds to under 2 seconds."

Q8 **What is CSRF and how does StayEase protect against it?**

"**CSRF (Cross-Site Request Forgery)** is an exploit where an unauthorized website tricks an authenticated user's browser into executing unwanted actions on your web application. StayEase defends against this using Spring Security's synchronized token pattern: every state-changing form (POST, PUT, DELETE) includes a unique, cryptographically generated CSRF token that is validated on the server."

Q9 **Why did you use Thymeleaf instead of a separate React frontend?**

"Thymeleaf provides seamless server-side rendering directly integrated with Spring MVC. It allows us to render secure, pre-computed HTML with zero flash of unstyled content, zero CORS configuration complexity, and natural templating (the HTML files can still be previewed in a browser without a build step). For an enterprise accommodation portal with security-sensitive bookings, server-side rendering also delivers superior SEO and fast initial load times."

Q10 **How do you know the project is reliable and bug-free?**

"The project is covered by a comprehensive automated test suite consisting of **329 automated unit and integration tests** built with JUnit 5, Mockito, and Spring Boot Test (`MockMvc`). The tests validate database models, security boundaries, role-based access rules, discovery endpoints, and location services. All 329 tests pass with 0 failures on every build."

Q11 **What is Docker's role in your project deployment?**

"Docker creates a standardized, containerized runtime environment. Our `Dockerfile` packages the Java 21 Eclipse Temurin JRE, the compiled application JAR, and the configuration. This completely eliminates the classic 'it works on my machine' problem, allowing Railway cloud to deploy the exact same image on its Linux edge servers as tested locally."

Q12 **If you had one more semester to work on this, what would you add next?**

"The next roadmap features would include:

1. **Real-Time Payment Gateway:** Integrating Razorpay or Stripe Webhooks for instant UPI, credit card, and net-banking transactions.
2. **Automated SMS & WhatsApp Alerts:** Sending instant check-in verification codes and booking receipts via Twilio or WhatsApp Business API.
3. **AI Accommodation Chatbot:** Integrating an LLM assistant using Spring AI to recommend stays based on student budget, college distance, and dietary preferences."

6. Quick Reference: Live URLs & Git Repository

Resource	Link / Location
Live Production App	`https://stayease-production-a18a.up.railway.app`
Worldwide Discovery Portal	`https://stayease-production-a18a.up.railway.app/discovery`
GitHub Repository	`https://github.com/ishangautam140675/StayEase`
Local Development URL	`http://localhost:8080/discovery`
Branch & Latest Commit	`main` (Fully committed & synced with origin)